



Today's agenda

↳ Kosaraju

↳ Prim's

↳ Kosaraju

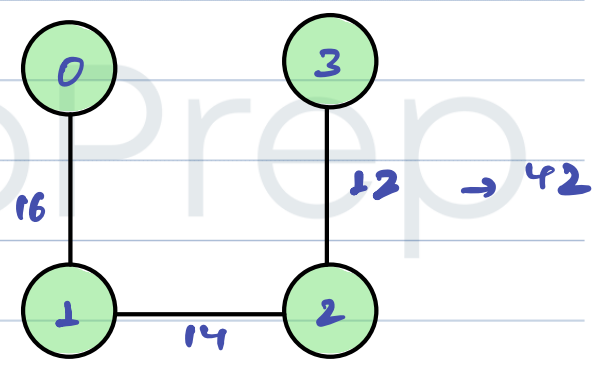
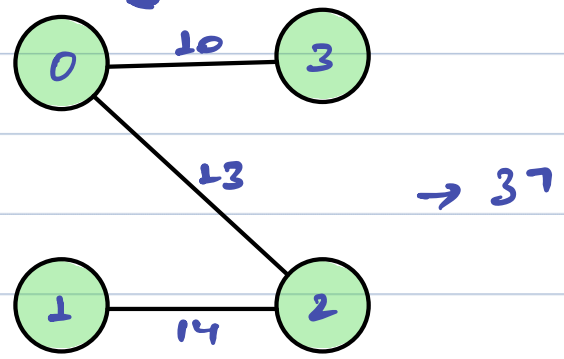
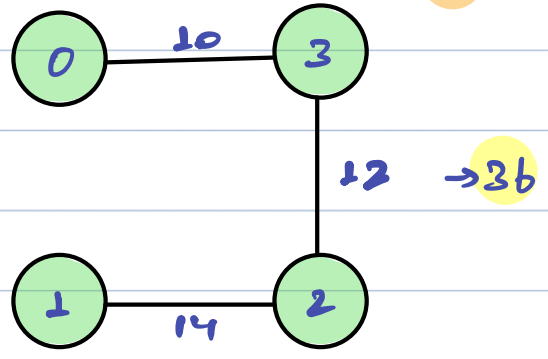
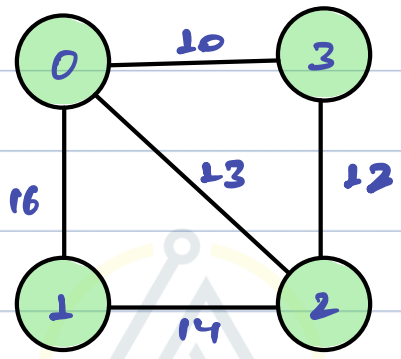


AlgoPrep



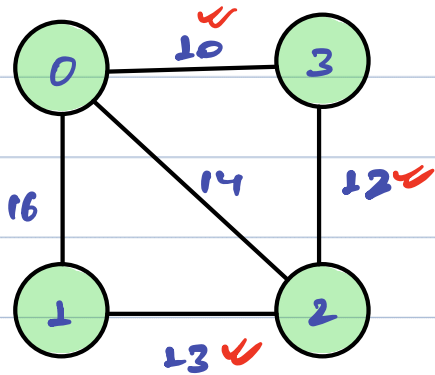
* MST $\rightarrow$ Minimum Spanning tree

- ① Convert graph into tree
 - $v \rightarrow$ vertices
 - $E = v - 1$
 - $\rightarrow$ 1 component only.
- ② choose minimum spanning tree out of all.





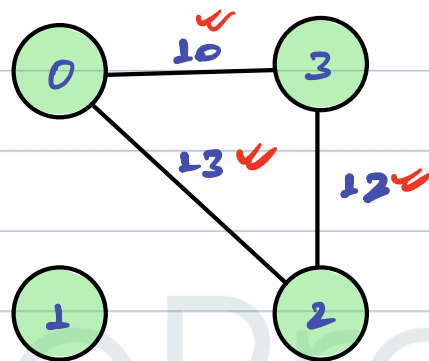
→ Find the weight of MST. → Kruskal's
→ Prim's



//idea! ~~XX~~ → incorrect idea

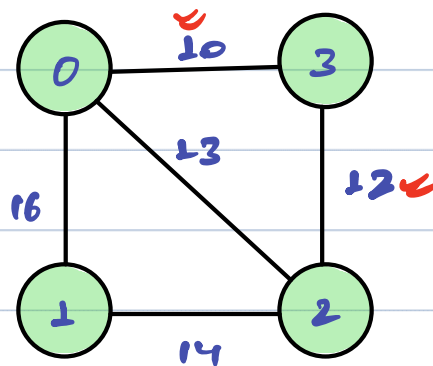
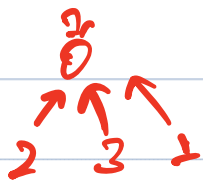
↳ Choose the $V-1$ edges in order of increasing value.

↳ ans = 35



AlgoPrep

//idea 2 → Kruskal



0	3	10
2	3	12
0	2	13 xx
1	2	14
0	1	16 xx

→

$$\text{ans} = 0 + 10 + 12 + 14 = 36$$



// Pseudo code

```

Public boolean union (int x, int y) {
    int Px = find(x);
    int Py = find(y);
    if (Px == Py) { return false; }
    if (rank[Px] > rank[Py]) {
        Par[Py] = Px;
    }
    else if (rank[Px] < rank[Py]) {
        Par[Px] = Py;
    }
    else { Par[Px] = Py; rank[Py]++; }
    return true;
}

```

```

Public int find (int x) {
    if (x == Par[x]) { return x; }
    int temp = find(Par[x]);
    Par[x] = temp;
    return temp;
}

```

```

int Parent[v];
int rank[v];

```

```

int kruskal (int v, int edges[E][3]) {
    Arrays.sort (edges, (a, b) -> Integer.compare(a[2], b[2]));
    for (int i=0; i<v; i++) {
        Parent[i] = i;
        rank[i] = 1;
    }
}

```

T.C: $O(E \log E) = O(E \log V)$
 S.C: $O(V)$

```

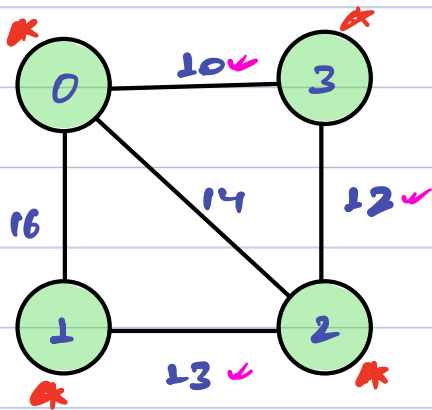
for (int i=0; i<E; i++) {
    int u = edges[i][0];
    int v = edges[i][1];
    int wt = edges[i][2];
    boolean flag = union(u, v);
    if (flag == true) { ans = ans + wt; }
}

return ans;
}

```



Idea 3 → Prim's algo



rem = { , }

~~{1,2}~~
~~{0,3}~~ ~~{2,3}~~
~~{1,3}~~
~~{3,0}~~
~~{2,1}~~

	0	1	2	3
vis	NT	NT	NT	NT

$$Ans = 0 + 0 + 10 + 12 + 13$$

T.C: $O(E \log E)$
 $= O(E \log V)$



// Pseudo code

```
class Pair1 { → Graph  
    int v;  
    int wt;  
}
```

```
class Pair2 { → PQ  
    int v + w;  
    int w;  
}
```

```
int[] Prim's (List<List<Pair1>> graph, int src) {
```

```
    PriorityQueue<Pair2> PQ;  
    int ans[N] = {∞};
```

```
    PQ.add(new Pair2(src, 0));
```

T.C: $O(E \log V)$
S.C: $O(E)$

```
    while (PQ.size() > 0) {
```

```
        Pair2 rem = PQ.remove();
```

```
        if (ans[rem.v + w] != ∞) { continue; }  
        ans[rem.v + w] = rem.w;
```

```
        List<Pair1> nbs = graph.get(rem.v + w);
```

```
        for (Pair1 n : nbs) {
```

```
            if (ans[n.v] != ∞) { continue; }
```

```
            PQ.add(new Pair2(n.v, rem.w + n.w));
```

```
    }
```

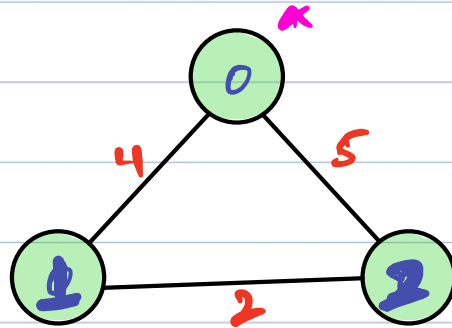
```
    }
```

```
    }
```

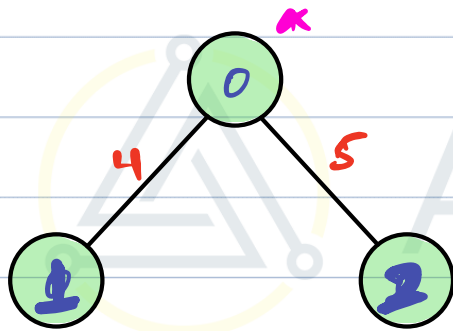
```
    for (int i = 0; i < N; i++) { sum = sum + ans[i]; } return sum;
```



Q) Prim's vs Dijkstra's

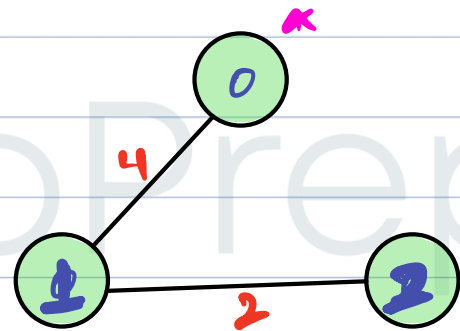


Shortest Path



- undirected graph.
- directed graph.

MST

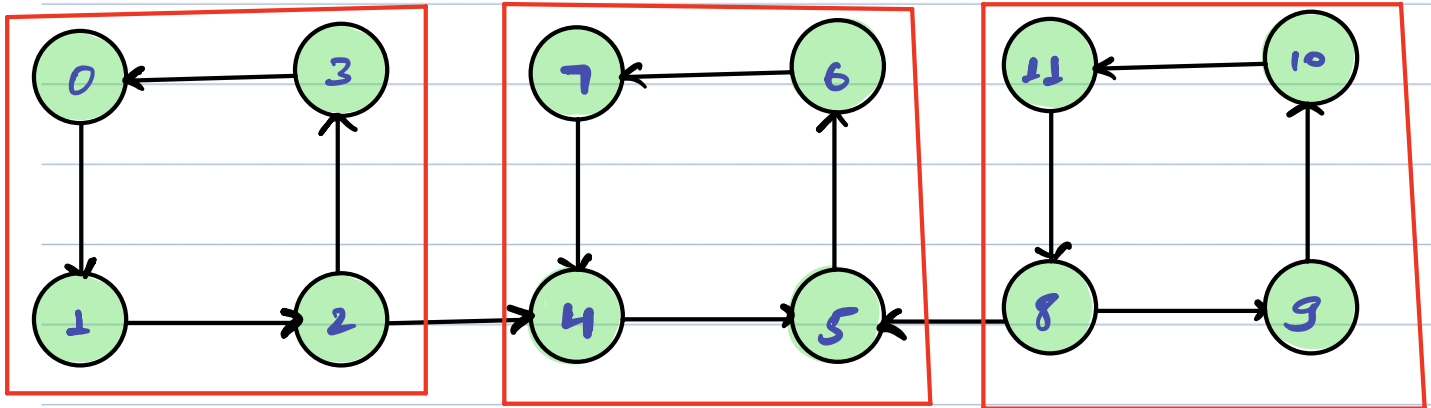


- undirected graph



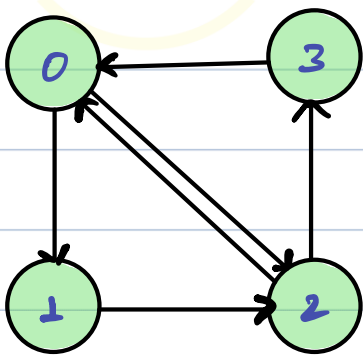
Q) Strongly Connected Component → Kosaraju

→ ans = 3



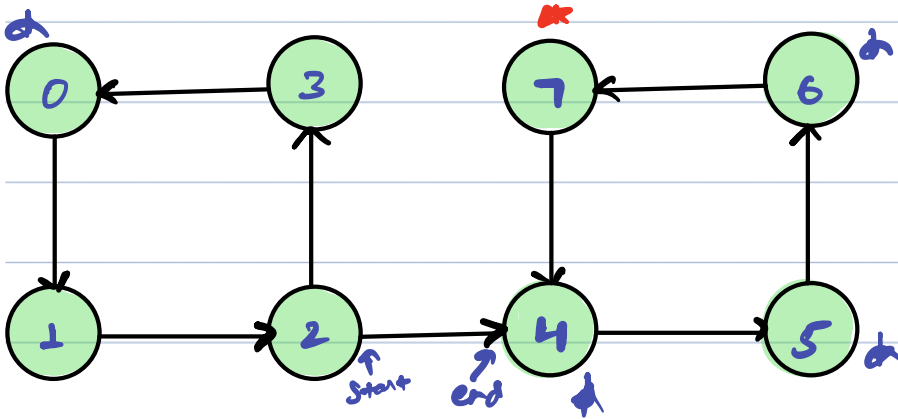
Idea 1 → Partially incorrect idea

↳ Count the No. of Cycle = No. of Strongly Connected Components

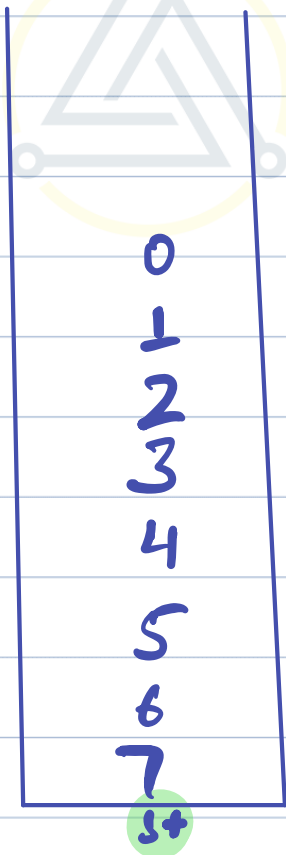




1/idea2 → Kosaraju algorithm



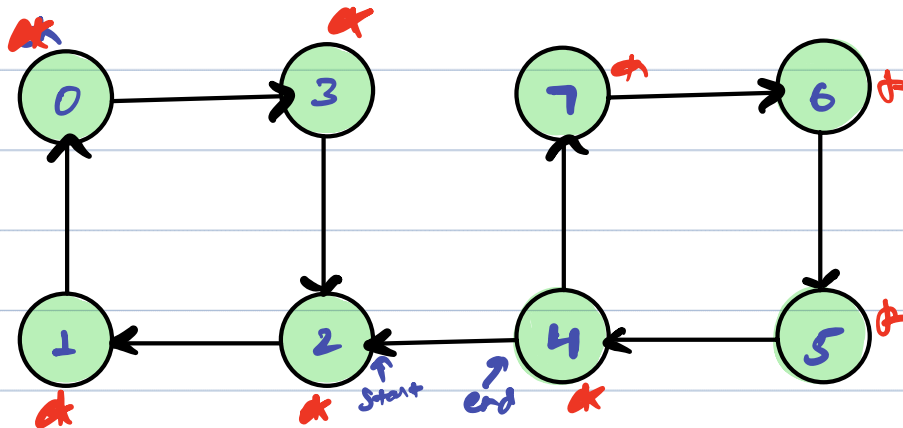
① creation of order in which we want to do d/s.



↳ The foremost element is going to be the element from the start side of problematic edge.



(ii) Reverse all the edges of graph.



$$ans = 0 + 1 + 1 = 2$$

~~0~~
~~1~~
~~2~~
~~3~~
~~4~~
~~5~~
~~6~~
~~7~~
8



//PSuedo code

```
int Kosaraju (List<List<Integer>> graph, int v){  
    //Step 1: Do dfs & store the vtn in stack while  
    boolean vis[v];  
    Stack<Integer> st;
```

```
    for (int i=0; i<v; i++){  
        if (vis[i]==false){  
            dfs(graph, vis, st, i);  
        }  
    }
```

T.C: $O(V+E)$
S.C: $O(V+E)$

// Step 2

Create a new graph with reversed edges.
↳ graph2

// Step 3

```
vis: new boolean [v];  
Stack<Integer> st2; //dummy stack  
int ans=0;
```

```
while (st.size()>0){  
    int rem = st.pop();  
    if (vis[rem]==false){  
        dfs(graph2, vis, st2, rem);  
        ans++;  
    }  
}  
return ans;
```

}



```
void dfs (List<List<Integer>> graph, boolean[] vis, Stack<Integer> st,  
int src)
```

```
    vis[src] = true;
```

```
    List<Integer> nbs = graph.get(src);
```

```
    for (int v : nbs) {  
        if (vis[v] == false) {  
            dfs(graph, vis, st, v);  
        }  
    }
```

```
    st.push(src);
```

```
}
```